

TW04: Functions, Packages and Loops

Five challenges | 20 marks each | 100 marks total

Student name	
Student ID	
Class / group	
Submission date	

Weekly purpose

Write reusable functions, use parameters and flexible arguments, understand scope and lambda expressions, and repeat work with loops.

SUBMISSION RULES

- Complete work in the corresponding Jupyter notebook.
- Write your own code and explanations; no solutions are supplied.
- Keep supplied data unchanged and work on copies.
- Restart the kernel and run all cells before submission.
- Submit the notebook with outputs visible and assistance acknowledged.

MARKING RUBRIC FOR EACH CHALLENGE

Assessment area	Marks
Correct method and working code	8
Validation and quality checks	4
Explanation and interpretation	4
Reproducibility and presentation	4
Total per challenge	20

Challenge 1: Write a reusable order-total function

Scenario

A shop needs one tested function instead of repeating a price calculation in several cells.

INSTRUCTIONS

1. Define `calculate_total` with required price, default quantity of 1, and default discount of 0.
2. Validate that price and quantity are not negative and discount is between 0 and 1.
3. Return the final amount instead of printing inside the function.
4. Call the function using positional and keyword arguments.
5. Test normal, default, and invalid inputs.

REQUIRED SUBMISSION

- A documented function with three parameters.
- Three valid test calls and one handled invalid case.
- A brief explanation of return versus print.

20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4

Challenge 2: Accept flexible inputs with args and kwargs

Scenario

A reporting function may receive any number of scores and optional display settings.

INSTRUCTIONS

1. Create `average_score(*scores)` and handle the empty-input case clearly.
2. Create `build_report(title, *values, **options)`.
3. Support `round_to` and `show_count` options from kwargs.
4. Ignore or report unexpected options instead of silently producing a wrong result.
5. Demonstrate both functions with at least three different calls.

REQUIRED SUBMISSION

- Two working flexible-argument functions.
- Three labelled example calls.
- A short explanation of the difference between `*args` and `**kwargs`.

20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4

Challenge 3: Sort records with lambda functions

Scenario

A manager wants the same product records ordered by different fields.

INSTRUCTIONS

1. Sort the supplied records by price from low to high using a lambda key.
2. Sort by rating from high to low.
3. Sort alphabetically by name without changing the original list.
4. Use a lambda with map or a comprehension to create VAT-inclusive prices.
5. Explain when a named function is clearer than a lambda.

REQUIRED SUBMISSION

- Three correctly sorted lists.
- A VAT-inclusive price result.
- Proof that the original order is unchanged and a clarity explanation.

20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4

Challenge 4: Control repetition with for and while loops

Scenario

A savings coach needs totals from monthly deposits and an estimate of months needed to reach a goal.

INSTRUCTIONS

1. Use a for loop to total the supplied deposits without sum().
2. Use enumerate to print month number and deposit.
3. Count deposits of at least 100 using a condition inside the loop.
4. Use a controlled while loop to estimate months needed to reach 1,000 with a regular 120 deposit.
5. Add a safety limit so the while loop cannot run forever.

REQUIRED SUBMISSION

- A correct accumulated total and numbered output.
- A correct count of qualifying deposits.
- A terminating while loop with a documented safety limit.

20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4

Challenge 5: Build a compact order analyser

Scenario

A small order list must be filtered and summarised by one reusable function.

INSTRUCTIONS

1. Use a list comprehension to select completed orders.
2. Use another comprehension to calculate revenue for each completed order.
3. Create a dictionary comprehension mapping order ID to revenue.
4. Write `summarise_orders(records)` that returns count, total revenue, and average revenue.
5. Test the function with the supplied records and an empty list.

REQUIRED SUBMISSION

- Two list comprehensions and one dictionary comprehension.
- A reusable function returning three summaries.
- Tests for normal and empty inputs.

20 marks: method and working code 8 | validation and quality checks 4 | explanation and interpretation 4 | reproducibility and presentation 4

Final submission checklist

- ☐ All five challenges are complete.
- ☐ The notebook runs from top to bottom without errors.
- ☐ Every output is clearly labelled.
- ☐ Important results include written explanations.
- ☐ Validation checks are included.
- ☐ Supplied raw data has not been overwritten.
- ☐ Sources and assistance are acknowledged.

Student declaration

I confirm that this submission is my work, that I have acknowledged sources and assistance, and that the displayed outputs were produced by the submitted code.

Signature: _____ Date: _____